

it is particularly suited for near-term quantum hardware.

QuTiP: A toolbox for simulating quantum systems, this language is useful for research and education.

Q#: Created by Microsoft, Q# is a standalone quantum language integrated with the Quantum Development Kit.

Other notable languages are PyQuil (Rigetti), Strawberry Fields (Xanadu), and Ocean (D-Wave), each of which targets specific hardware or quantum computing models. These languages make quantum programming more accessible, abstracting complex physics and allowing programmers to focus on logic and algorithms.

Coding with Qiskit and Cirq

To illustrate quantum programming, let's examine basic examples with Qiskit and Cirq. Both frameworks let you define quantum circuits, run simulations, and analyse results. While their syntax differs, the underlying ideas are similar.

Example 1: Creating a quantum circuit in Qiskit

```
from qiskit import QuantumCircuit, Aer, execute
# Create a simple quantum circuit with one qubit and one
classical bit
qc = QuantumCircuit(1, 1)
qc.h(0)      # Apply Hadamard gate to put the qubit in
superposition
qc.measure(0, 0) # Measure the qubit
# Simulate the circuit
simulator = Aer.get_backend('qasm_simulator')
result = execute(qc, simulator, shots=1000).result()
counts = result.get_counts(qc)
print("Measurement results:", counts)
```

This code creates a single qubit, applies a Hadamard gate to achieve superposition, and then measures the result. The output shows how often each result (0 or 1) occurred over 1,000 runs.

Example 2: Building the same circuit in Cirq

```
import cirq
# Create a qubit
qubit = cirq.LineQubit(0)
circuit = cirq.Circuit()
# Apply Hadamard gate and measure
circuit.append([cirq.H(qubit), cirq.measure(qubit, key='m')])
# Simulate the circuit
simulator = cirq.Simulator()
result = simulator.run(circuit, repetitions=1000)
print("Measurement results:", result.histogram(key='m'))
```

The sample output using a quantum simulator is:

Measurement results: {0: 513, 1: 487}

In this output, the measurement results show approximately equal counts for 0 and 1, reflecting the probabilistic nature of quantum measurement after applying a Hadamard gate to the qubit. The slight difference in counts is due to statistical variation over 1000 repetitions, as expected from an ideal quantum simulation.

In Cirq, the approach is similar — to define a qubit, apply a Hadamard gate, and measure. The result is a histogram of measurement outcomes, showing the quantum nature of computation.

Beyond Qiskit and Cirq, other languages offer unique features, but the essential building blocks—creating circuits, applying gates, and measuring—remain central.

Simulators and emulators: Testing code on classical computers

Access to real quantum hardware is limited and often expensive. Simulators and emulators bridge the gap by allowing quantum code to be tested on classical computers. Simulators model ideal quantum behaviour, while emulators may introduce noise to mimic real hardware.

Both Qiskit and Cirq come with built-in simulators. For example, the code snippets above use Qiskit's `qasm_simulator` and Cirq's simulator. These tools help programmers develop and debug algorithms before deploying them on actual quantum processors.

Here's an example of noisy simulation in Cirq:

```
import cirq
import numpy as np
qubit = cirq.LineQubit(0)
circuit = cirq.Circuit(cirq.H(qubit), cirq.measure(qubit,
key='m'))
# Add noise
noisy_simulator = cirq.DensityMatrixSimulator(noise=cirq.
depolarize(p=0.1))
result = noisy_simulator.run(circuit, repetitions=1000)
print("Noisy measurement results:", result.histogram(key='m'))
```

When you run the above Cirq code with the depolarising noise using the `DensityMatrixSimulator`, you will get an output like this:

Noisy measurement results: {0: 545, 1: 455}

This output shows the histogram of measurement results for the qubit after 1,000 repetitions, where the key '0' corresponds to measuring the qubit in the $|0\rangle$ state and '1' to the $|1\rangle$ state. The results will vary slightly with each run due to the inherent randomness and simulated noise. This example